



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش پنجم

طراحی ضرب‌کننده (روش Booth)

خلاصه

در این آزمایش یک واحد ضرب‌کننده‌ی علامت‌دار با روش بوث (Booth) طراحی شده است. طبق خواسته‌ی صورت آزمایش، مسیر داده (datapath) و واحد کنترل (control unit) به صورت جداگانه طراحی و سپس در یک ماژول سطح بالا به یکدیگر متصل شده‌اند.

نکته‌ی کلیدی صورت آزمایش این است که واحد شیفت‌دهنده باید بتواند بیش از یک بیت در هر پالس ساعت شیفت بدهد تا نسبت به ضرب عادی (Shift & Add) تسریع داشته باشد. برای برآورده کردن این شرط، از بوث پایه ۴ (radix-4 یا modified Booth) استفاده شده است که در هر تکرار دو بیت از مضروب‌فیه را مصرف می‌کند و مجموعه‌ی $\{A, Q, Q_{-1}\}$ را دو بیت به راست شیفت می‌دهد. نتیجه این است که برای عملوند N بیتی به جای N تکرار، تنها $N/2$ تکرار لازم است.

صحت‌سنجی با Icarus Verilog انجام شده و ضرب‌کننده روی تمام فضای ورودی علامت‌دار ۸ در ۸ بیت (هر $256 \times 256 = 65536$ جفت) با عملگر ضرب علامت‌دار خود Verilog مقایسه شده است.

چرا پایه ۴؟

در روش Shift & Add معمولی و همچنین بوث پایه ۲، در هر پالس ساعت تنها یک بیت از مضروب‌فیه بررسی و یک بیت شیفت انجام می‌شود؛ پس ضرب دو عدد N بیتی N کلاک طول می‌کشد.

در بوث پایه ۴، در هر تکرار یک پنجره‌ی سه‌بیتی از $\{Q_1, Q_0, Q_{-1}\}$ بررسی می‌شود و بر اساس آن یکی از پنج مضرب $\{0, +M, -M, +2M, -2M\}$ به انباشتگر اضافه می‌شود، سپس شیفت دو بیتی انجام می‌گیرد. جدول بازکدگذاری (recoding) به این صورت است:

تفسیر	عملیات	Q_{-1}	Q_0	Q_1
دنباله‌ی صفر	$A += 0$	۰	۰	۰
پایان دنباله‌ی یک	$A += M$	۱	۰	۰
یک تکی	$A += M$	۰	۱	۰
پایان دنباله‌ی یک	$A += 2M$	۱	۱	۰
آغاز دنباله‌ی یک	$A -= 2M$	۰	۰	۱
صفر تکی	$A -= M$	۱	۰	۱
آغاز دنباله‌ی یک	$A -= M$	۰	۱	۱
دنباله‌ی یک	$A += 0$	۱	۱	۱

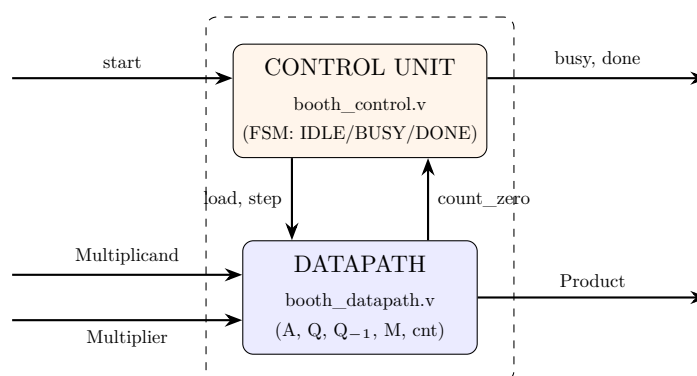
مقایسه‌ی تعداد کلاک‌ها که در تست‌بنچ هم اندازه‌گیری شده است:

N	پایه ۲ / Shift & Add	پایه ۴ (این طراحی)	تسریع
۴	۵ کلاک	۳ کلاک	$1/7 \times$
۸	۹ کلاک	۵ کلاک	$1/8 \times$
۱۶	۱۷ کلاک	۹ کلاک	$1/9 \times$

اعداد ستون سوم مستقیماً از خود مدار خوانده شده‌اند، نه اینکه فرض شده باشند؛ تست‌بنچ لبه‌های کلاک را تا فعال شدن done می‌شمارد.

معماری کلی

طراحی از سه ماژول تشکیل شده است:



ماژول	مسئولیت
booth_datapath.v	همه‌ی رجیسترها و محاسبات: انباشتگر A ، مضروب Q ، بیت Q_{-1} ، مضروب M ، شمارنده، بازکدگذاری بوث و شیفت دوبیتی. هیچ تصمیم ترتیبی نمی‌گیرد.
booth_control.v	ماشین حالت Moore با سه حالت. فقط تعیین می‌کند چه زمانی عملوندها بارگذاری و چه زمانی یک تکرار اجرا شود. هیچ داده‌ای نگه نمی‌دارد.
booth_multiplier.v	اتصال این دو به یکدیگر؛ خودش منطقی ندارد.

مشخصات واسط

سیگنال	جهت	پهنای	توضیح
Clk	ورودی	۱	کلاک؛ لبه‌ی بالارونده
RstN	ورودی	۱	ریست ناهم‌گام فعال پایین
start	ورودی	۱	درخواست شروع یک ضرب جدید
Multiplicand	ورودی	N	مضروب (علامت‌دار، مکمل ۲)
Multiplier	ورودی	N	مضروب‌فیه (علامت‌دار، مکمل ۲)
Product	خروجی	$2N$	حاصل ضرب علامت‌دار
busy	خروجی	۱	یعنی ضرب در جریان است
done	خروجی	۱	یعنی نتیجه آماده است

سیگنال done از نوع سطح است و تا رسیدن start بعدی بالا می‌ماند، بنابراین یک مصرف‌کننده‌ی کند هم نتیجه را از دست نمی‌دهد.

مسیر داده

نکته‌ای که در پیاده‌سازی اهمیت زیادی دارد پهنای انباشتگر است. در بسیاری از توضیحات درسی، A را $N+1$ بیت می‌گیرند؛ اما در پایه ۴ این کافی نیست. اگر مضروب برابر منفی‌ترین مقدار ممکن یعنی $M = -2^{N-1}$ باشد، آنگاه:

$$2M = -2^N \quad \text{و} \quad -2M = +2^N$$

مقدار $+2^N$ در $N+1$ بیت علامت‌دار جا نمی‌شود (بازه‌ی آن $[-2^N, 2^N - 1]$ است). بنابراین در این طراحی A و مقدار افزودنی هر دو $N+2$ بیت گرفته شده‌اند تا هر پنج مضرب دقیقاً نمایش‌پذیر باشند و منفی‌ترین عملوند نیازی به حالت خاص نداشته باشد.

```

1 module booth_datapath #(
2     parameter integer N = 8
3 ) (
4     input wire          Clk,
5     input wire          RstN,
6     input wire          load,
7     input wire          step,
8     input wire signed [N-1:0] Multiplicand,
9     input wire signed [N-1:0] Multiplier,
10    output wire [N:0] count,
11    output wire          count_zero,
12    output wire signed [2*N-1:0] Product
13 );
14 localparam integer ITER = N / 2;
15 localparam integer CNT_W = $clog2(N/2) + 1;
16 reg signed [N+1:0] A;
17 reg signed [N-1:0] Q;
18 reg signed [N-1:0] Q_1;
19 reg signed [N-1:0] M;
20 reg signed [CNT_W-1:0] cnt;
21
22 assign count = cnt;
23 assign count_zero = (cnt == 0);
24 assign Product = {A[N-1:0], Q};
25 wire [2:0] booth_window = {Q[1], Q[0], Q_1};
26 wire signed [N+1:0] M_ext = {{2{M[N-1]}}, M};
27 wire signed [N+1:0] M2_ext = {M[N-1], M, 1'b0};
28 reg signed [N+1:0] addend;
29 always @(*) begin
30     case (booth_window)
31         3'b000, 3'b111: addend = {(N+2){1'b0}};
32         3'b001, 3'b010: addend = M_ext;
33         3'b011: addend = M2_ext;
34         3'b100: addend = -M2_ext;
35         3'b101, 3'b110: addend = -M_ext;
36         default: addend = {(N+2){1'b0}};
37     endcase
38 end
39
40 wire signed [N+1:0] A_sum = A + addend;
41 wire signed [N+1:0] A_next = {A_sum[N+1], A_sum[N+1], A_sum[N+1:2]};
42 wire signed [N-1:0] Q_next = {A_sum[1:0], Q[N-1:2]};
43 wire signed [N-1:0] Q_1_next = Q[1];
44
45 always @(posedge Clk or negedge RstN) begin
46     if (!RstN) begin
47         A <= {(N+2){1'b0}};
48         Q <= {N{1'b0}};
49         Q_1 <= 1'b0;
50         M <= {N{1'b0}};
51         cnt <= {CNT_W{1'b0}};
52     end else if (load) begin
53         A <= {(N+2){1'b0}};
54         Q <= Multiplier;
55         Q_1 <= 1'b0;
56         M <= Multiplicand;
57         cnt <= ITER[CNT_W-1:0];
58     end else if (step && cnt != 0) begin
59         A <= A_next;
60         Q <= Q_next;
61         Q_1 <= Q_1_next;
62         cnt <= cnt - 1'b1;

```

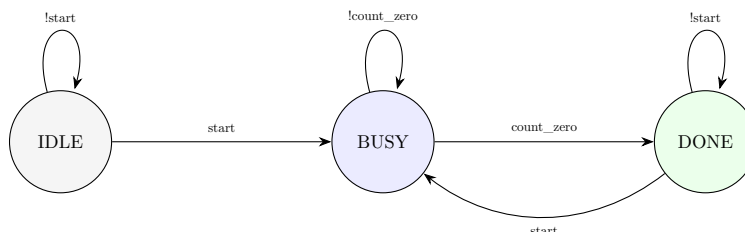
```

63     end
64     end
65 endmodule

```

واحد کنترل

واحد کنترل یک ماشین حالت Moore با سه حالت است:



حالت	load	step	توضیح
IDLE	start	۰	منتظر درخواست؛ با start عملوندها بارگذاری می‌شوند
BUSY	۰	۱	در هر کلاک یک تکرار پایه ۴ اجرا می‌شود
DONE	start	۰	نتیجه معتبر است و تا start بعدی می‌ماند

توجه کنید که واحد کنترل خودش شمارنده ندارد؛ شمارنده در مسیرهاده است و کنترل فقط پرچم وضعیت count_zero را می‌بیند. این تفکیک باعث می‌شود تغییر عمق تکرار (مثلا با تغییر N) هیچ اثری روی واحد کنترل نداشته باشد.

```

1 module booth_control #(
2     parameter integer N = 8
3 ) (
4     input wire Clk,
5     input wire RstN,
6     input wire start,
7     input wire count_zero,
8     output reg load,
9     output reg step,
10    output wire busy,
11    output wire done
12 );
13    localparam [1:0] S_IDLE = 2'd0,
14                  S_BUSY = 2'd1,
15                  S_DONE = 2'd2;
16
17    reg [1:0] state, next_state;
18
19    always @(posedge Clk or negedge RstN) begin
20        if (!RstN) state <= S_IDLE;
21        else state <= next_state;
22    end
23
24    always @(*) begin
25        case (state)
26            S_IDLE: next_state = start ? S_BUSY : S_IDLE;
27            S_BUSY: next_state = count_zero ? S_DONE : S_BUSY;
28            S_DONE: next_state = start ? S_BUSY : S_DONE;
29            default: next_state = S_IDLE;
30        endcase
31    end
32
33    always @(*) begin
34        load = 1'b0;
35        step = 1'b0;
36        case (state)
37            S_IDLE: load = start;
38            S_BUSY: step = 1'b1;
39            S_DONE: load = start;

```

```

40         default: ;
41     endcase
42 end
43
44 assign busy = (state == S_BUSY);
45 assign done = (state == S_DONE);
46 endmodule

```

اتصال دو واحد

ماژول سطح بالا فقط این دو را به هم وصل می‌کند و منطق مستقلی ندارد:

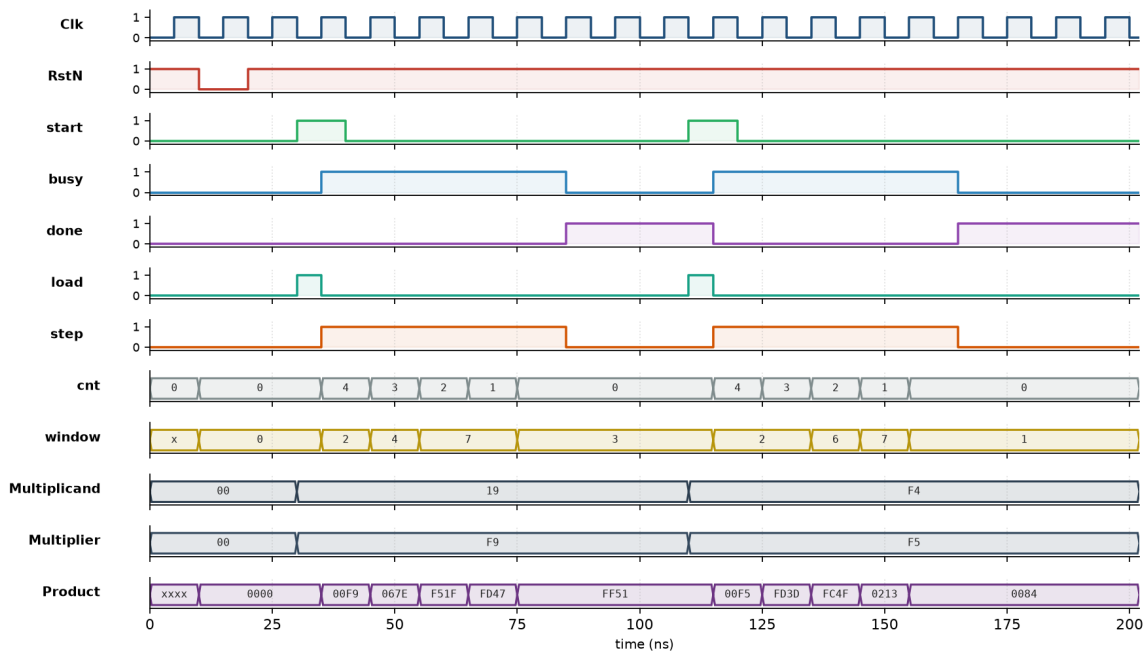
```

1 module booth_multiplier #(
2     parameter integer N = 8
3 ) (
4     input wire          Clk,
5     input wire          RstN,
6     input wire          start,
7     input wire signed [N-1:0] Multiplicand,
8     input wire signed [N-1:0] Multiplier,
9     output wire signed [2*N-1:0] Product,
10    output wire          busy,
11    output wire          done
12 );
13 wire load, step, count_zero;
14
15 booth_control #(.N(N)) u_control (
16     .Clk          (Clk),
17     .RstN         (RstN),
18     .start        (start),
19     .count_zero   (count_zero),
20     .load         (load),
21     .step         (step),
22     .busy         (busy),
23     .done         (done)
24 );
25
26 booth_datapath #(.N(N)) u_datapath (
27     .Clk          (Clk),
28     .RstN         (RstN),
29     .load         (load),
30     .step         (step),
31     .Multiplicand (Multiplicand),
32     .Multiplier   (Multiplier),
33     .count        (),
34     .count_zero   (count_zero),
35     .Product       (Product)
36 );
37 endmodule

```

شکل موج

شکل موج دو ضرب پشت سرهم را نشان می‌دهد: ابتدا $-175 = -7 \times 25$ و سپس $+132 = (-11) \times (-12)$ ؛ یعنی هر دو حالت افزودن و کاستن در بازکدگذاری بوث دیده می‌شود.

Radix-4 Booth multiplier (N=8) - $25 \times (-7) = -175$, then $(-12) \times (-11) = +132$; 5 clocks each

نکات قابل مشاهده در شکل: شمارنده‌ی cnt دنباله‌ی $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ را طی می‌کند، یعنی دقیقاً چهار تکرار برای عملوند ۸ بیتی و نه هشت تکرار. سیگنال window همان پنجره‌ی سه‌بیتی است که در هر تکرار عوض می‌شود. مقدار Product در پایان ضرب اول برابر FF51 است که در مکمل ۲ همان ۱۷۵- می‌شود، و در پایان ضرب دوم 0084 یعنی ۱۳۲+. همچنین دیده می‌شود که load تنها یک پالس تک‌کلاکی است در حالی که step در سراسر حالت BUSY فعال می‌ماند.

تست بنچ اصلی

تست بنچ اصلی حاصل ضرب را با عملگر ضرب علامت‌دار خود Verilog مقایسه می‌کند و کل فضای ورودی ۸ در ۸ بیت را به صورت exhaustive می‌پیماید. علاوه بر آن، حالت‌های مرزی کلاسیک به صورت جداگانه هم آزموده شده‌اند: صفرها، ± 1 ، بزرگ‌ترین مثبت‌ها، منفی‌ترین مقدار -128 (از جمله -128×-128)، الگوهای متناوب $0xAA/0x55$ ، شروع‌های پشت‌سرهم بدون ریست، و ریست در میانه‌ی یک ضرب.

```

1 `timescale 1ns/1ps
2
3 module tb_booth_multiplier;
4     localparam integer N = 8;
5
6     reg          Clk, RstN, start;
7     reg signed [N-1:0] Multiplicand, Multiplier;
8     wire signed [2*N-1:0] Product;
9     wire          busy, done;
10
11     booth_multiplier #(N(N)) dut (
12         .Clk(Clk), .RstN(RstN), .start(start),
13         .Multiplicand(Multiplicand), .Multiplier(Multiplier),
14         .Product(Product), .busy(busy), .done(done)
15     );
16
17     integer pass, fail;
18     integer ai, bi;
19     integer cycles, max_cycles, min_cycles;
20     reg signed [2*N-1:0] expected;
21     reg signed [N-1:0] sa, sb;
22
23     initial Clk = 1'b0;
24     always #5 Clk = ~Clk;
25
26     task automatic multiply;

```

```

27   input signed [N-1:0] a;
28   input signed [N-1:0] b;
29   begin
30       @(negedge Clk);
31       Multiplicand = a;
32       Multiplier   = b;
33       start        = 1'b1;
34       @(negedge Clk);
35       start = 1'b0;
36
37       cycles = 0;
38       while (!done) begin
39           @(posedge Clk); #1;
40           cycles = cycles + 1;
41           if (cycles > 100) begin
42               $display("FAIL: timeout waiting for done (a=%0d b=%0d)", a, b);
43               $fatal(1);
44           end
45       end
46       #1;
47   end
48 endtask
49
50 task automatic check;
51     input signed [N-1:0] a;
52     input signed [N-1:0] b;
53     begin
54         multiply(a, b);
55         expected = a * b;
56         if (Product == expected) begin
57             pass = pass + 1;
58         end else begin
59             $display("FAIL a=%0d b=%0d got=%0d (%h) exp=%0d (%h)",
60                 a, b, Product, Product, expected, expected);
61             fail = fail + 1;
62         end
63         if (cycles > max_cycles) max_cycles = cycles;
64         if (cycles < min_cycles) min_cycles = cycles;
65     end
66 endtask
67
68 task automatic do_reset;
69     begin
70         @(negedge Clk);
71         start = 1'b0; Multiplicand = 0; Multiplier = 0;
72         RstN = 1'b0;
73         @(negedge Clk);
74         RstN = 1'b1;
75         #1;
76         if (busy != 1'b0 || done != 1'b0) begin
77             $display("FAIL: flags not clear after reset (busy=%b done=%b)",
78                 busy, done);
79             fail = fail + 1;
80         end else pass = pass + 1;
81     end
82 endtask
83
84 initial begin
85     pass = 0; fail = 0;
86     max_cycles = 0; min_cycles = 1000;
87     RstN = 1'b1; start = 0; Multiplicand = 0; Multiplier = 0;
88
89     do_reset;
90
91     check( 0, 0);
92     check( 0, 57);
93     check( 57, 0);
94     check( 1, 1);
95     check( 1, -1);
96     check(-1, 1);
97     check(-1, -1);
98     check(127, 127);
99     check(127, -128);
100    check(-128, 127);
101    check(-128, -128);
102    check(-128, 1);
103    check( 1, -128);
104    check(-128, -1);
105    check(-1, -128);
106    check( 85, -86);
107    check(-86, 85);
108    check( 85, 85);
109    check(-86, -86);
110
111    check( 12, 10);
112    check(-12, 10);
113    check( 12, -10);

```

```

114     check(-12, -10);
115
116     @(negedge Clk);
117     Multiplicand = 100; Multiplier = 100; start = 1'b1;
118     @(negedge Clk); start = 1'b0;
119     @(posedge Clk);
120     @(posedge Clk);
121     do_reset;
122     check(6, 7);
123
124     for (ai = -128; ai <= 127; ai = ai + 1)
125         for (bi = -128; bi <= 127; bi = bi + 1) begin
126             sa = ai[N-1:0];
127             sb = bi[N-1:0];
128             check(sa, sb);
129         end
130
131     $display("cycles per multiply: min=%0d max=%0d (N=%0d, radix-4 => %0d iterations)",
132             min_cycles, max_cycles, N, N/2);
133     $display("tb_booth_multiplier: Passed: %0d, Failed: %0d", pass, fail);
134     if (fail != 0) $fatal(1);
135     $finish;
136 end
137 endmodule

```

تست‌بنج پارامتری و اندازه‌گیری تسریع

تست‌بنج دوم دو کار انجام می‌دهد که تست اول انجام نمی‌دهد. اول اینکه همان ماژول را با سه پهنای متفاوت ($N = 4, 8, 16$) می‌سازد تا مطمئن شویم هیچ‌جای طراحی به عدد ۸ گره نخورده است. دوم اینکه تعداد کلاک هر ضرب را از خود مدار اندازه می‌گیرد و بررسی می‌کند که واقعا برابر $N/2 + 1$ باشد؛ یعنی ادعای شیفیت بیش از یک بیت در هر کلاک به‌صورت عددی اثبات می‌شود نه اینکه صرفا ادعا شود.

```

1 `timescale 1ns/1ps
2
3 module tb_booth_param;
4
5     integer pass, fail;
6
7     // ----- N = 4 -----
8     reg          Clk4, RstN4, start4;
9     reg signed [3:0] a4, b4;
10    wire signed [7:0] p4;
11    wire          busy4, done4;
12    booth_multiplier #(N(4)) dut4 (
13        .Clk(Clk4), .RstN(RstN4), .start(start4),
14        .Multiplicand(a4), .Multiplier(b4),
15        .Product(p4), .busy(busy4), .done(done4));
16    initial Clk4 = 0;
17    always #5 Clk4 = ~Clk4;
18
19    // ----- N = 8 -----
20    reg          Clk8, RstN8, start8;
21    reg signed [7:0] a8, b8;
22    wire signed [15:0] p8;
23    wire          busy8, done8;
24    booth_multiplier #(N(8)) dut8 (
25        .Clk(Clk8), .RstN(RstN8), .start(start8),
26        .Multiplicand(a8), .Multiplier(b8),
27        .Product(p8), .busy(busy8), .done(done8));
28    initial Clk8 = 0;
29    always #5 Clk8 = ~Clk8;
30
31    // ----- N = 16 -----
32    reg          Clk16, RstN16, start16;
33    reg signed [15:0] a16, b16;
34    wire signed [31:0] p16;
35    wire          busy16, done16;
36    booth_multiplier #(N(16)) dut16 (
37        .Clk(Clk16), .RstN(RstN16), .start(start16),
38        .Multiplicand(a16), .Multiplier(b16),
39        .Product(p16), .busy(busy16), .done(done16));
40    initial Clk16 = 0;
41    always #5 Clk16 = ~Clk16;
42
43    integer cyc4, cyc8, cyc16;
44    integer i, j;
45
46    reg signed [3:0] sa4, sb4;
47    reg signed [7:0] sa8, sb8;

```

```

48 reg signed [15:0] sa16, sb16;
49 reg signed [7:0] exp4;
50 reg signed [15:0] exp8;
51 reg signed [31:0] exp16;
52
53 task automatic report;
54     input [255:0] tag;
55     input ok;
56     begin
57         if (ok) pass = pass + 1;
58         else begin
59             $display("FAIL [%0s]", tag);
60             fail = fail + 1;
61         end
62     end
63 endtask
64
65 // --- N=4 : exhaustive over all signed 4-bit pairs ---
66 task automatic run4;
67     begin
68         @(negedge Clk4); RstN4 = 0; start4 = 0; a4 = 0; b4 = 0;
69         @(negedge Clk4); RstN4 = 1;
70         for (i = -8; i <= 7; i = i + 1) begin
71             for (j = -8; j <= 7; j = j + 1) begin
72                 @(negedge Clk4);
73                 sa4 = i[3:0]; sb4 = j[3:0]; exp4 = sa4 * sb4;
74                 a4 = sa4; b4 = sb4; start4 = 1;
75                 @(negedge Clk4); start4 = 0;
76                 cyc4 = 0;
77                 while (!done4) begin
78                     @(posedge Clk4); #1; cyc4 = cyc4 + 1;
79                 end
80                 #1;
81                 report("N4/product", p4 === exp4);
82             end
83         end
84         report("N4/cycles", cyc4 == (4/2) + 1);
85         $display(" N=4 : %0d clocks per multiply (radix-2 would need %0d)",
86                 cyc4, 4 + 1);
87     end
88 endtask
89
90 // --- N=8 : sampled pairs incl. the most-negative value ---
91 task automatic run8;
92     begin
93         @(negedge Clk8); RstN8 = 0; start8 = 0; a8 = 0; b8 = 0;
94         @(negedge Clk8); RstN8 = 1;
95         for (i = -128; i <= 127; i = i + 17) begin
96             for (j = -128; j <= 127; j = j + 13) begin
97                 @(negedge Clk8);
98                 sa8 = i[7:0]; sb8 = j[7:0]; exp8 = sa8 * sb8;
99                 a8 = sa8; b8 = sb8; start8 = 1;
100                 @(negedge Clk8); start8 = 0;
101                 cyc8 = 0;
102                 while (!done8) begin
103                     @(posedge Clk8); #1; cyc8 = cyc8 + 1;
104                 end
105                 #1;
106                 report("N8/product", p8 === exp8);
107             end
108         end
109         report("N8/cycles", cyc8 == (8/2) + 1);
110         $display(" N=8 : %0d clocks per multiply (radix-2 would need %0d)",
111                 cyc8, 8 + 1);
112     end
113 endtask
114
115 task automatic run16;
116     begin
117         @(negedge Clk16); RstN16 = 0; start16 = 0; a16 = 0; b16 = 0;
118         @(negedge Clk16); RstN16 = 1;
119         for (i = -32768; i <= 32767; i = i + 4447) begin
120             for (j = -32768; j <= 32767; j = j + 5119) begin
121                 @(negedge Clk16);
122                 sa16 = i[15:0]; sb16 = j[15:0]; exp16 = sa16 * sb16;
123                 a16 = sa16; b16 = sb16; start16 = 1;
124                 @(negedge Clk16); start16 = 0;
125                 cyc16 = 0;
126                 while (!done16) begin
127                     @(posedge Clk16); #1; cyc16 = cyc16 + 1;
128                 end
129                 #1;
130                 report("N16/product", p16 === exp16);
131             end
132         end
133         report("N16/cycles", cyc16 == (16/2) + 1);
134         $display(" N=16 : %0d clocks per multiply (radix-2 would need %0d)",

```

```

135         cyc16, 16 + 1);
136     end
137 endtask
138
139 initial begin
140     pass = 0; fail = 0;
141     RstN4 = 1; RstN8 = 1; RstN16 = 1;
142     start4 = 0; start8 = 0; start16 = 0;
143     $display("measured iteration counts:");
144     run4;
145     run8;
146     run16;
147     $display("tb_booth_param: Passed: %0d, Failed: %0d", pass, fail);
148     if (fail != 0) $fatal(1);
149     $finish;
150 end
151 endmodule

```

نتایج تست

```

=== tb_booth_multiplier ===
cycles per multiply: min=5 max=5 (N=8, radix-4 => 4 iterations)
tb_booth_multiplier: Passed: 65562, Failed: 0
PASS: tb_booth_multiplier

=== tb_booth_param ===
measured iteration counts:
  N=4 : 3 clocks per multiply (radix-2 would need 5)
  N=8 : 5 clocks per multiply (radix-2 would need 9)
  N=16 : 9 clocks per multiply (radix-2 would need 17)
tb_booth_param: Passed: 774, Failed: 0
PASS: tb_booth_param

summary: 2/2 passed, 0 failed

```

جمع‌بندی

در این آزمایش دیدیم که تفکیک مسیر داده از واحد کنترل چه فایده‌ای دارد. مسیر داده فقط می‌داند چگونه یک تکرار بوث را انجام دهد و واحد کنترل فقط می‌داند چند بار و چه زمانی این کار باید تکرار شود. نتیجه این است که هیچ‌کدام به جزئیات دیگری وابسته نیستند؛ مثلاً تغییر N از ۸ به ۱۶ فقط تعداد تکرارها را عوض می‌کند و یک خط از واحد کنترل هم تغییر نمی‌کند.

درباره‌ی شرط تسریع، استفاده از بوث پایه ۴ باعث شد تعداد کلاک‌ها از $N + 1$ به $N/2 + 1$ کاهش پیدا کند، یعنی حدود دو برابر سریع‌تر. البته این تسریع رایگان نیست: در ازای آن باید یک جمع‌کننده‌ی پهن‌تر و منطق انتخاب پنج‌حالتی برای ضرب‌ها پرداخت شود، که مصالحه‌ی همیشگی سرعت در برابر سطح است.

خروجی سنتز (Quartus Prime)

طرح روی Intel Quartus Prime Lite Edition 21.1.0 با موجودیت سطح بالای booth_multiplier سنتز و Place & Route شد. دستگاه هدف: Cyclone V (5CGXFC7C7F23C8).

مورد	نتیجه
وضعیت کامپایل	موفق (۰ خطا، ۱۳ هشدار)
Logic utilization (ALMs)	۲۸/۵۶۴۸۰ (< ۱٪)
Total registers	۳۰
Total pins	۳۷/۲۶۸ (۱۴٪)
Block memory bits	•
DSP Blocks	•

